به نام حق

علی سپهرنیا (۹۶۱۲۱۰۳۸) گزارش کار آزمایشگاه سیستم دیجیتال ۲ (جلسه پنجم) استاد: خانم رضایی

اندازه گیری دما توسط ATMEGA۱۶

برنامه ای بنویسید که دمای محیط را اندازه گیری و روی LCD کاراکتری نمایش داده شود.

شرح گزارش کار :

I. در کدویژن یک پروژه جدید ایجاد میکنیم. در پنجره کد ویزارد چیپ ATMEGA16 را انتخاب میکنیم. برای اندازه گیری دما نیاز است که از واحد تبدیل آنالوگ به دیجیتال (ADC) میکروکنترلر استفاده کنیم. بنابراین به تب Analog to Digital Converter می رویم و آن را فعال میکنیم. واحد ADC دارای دو حالت وضوح ۸ بیتی و ۱۰ بیتی است. ما از حالت ۸ بیتی استفاده می کنیم پس گزینه Use 8 bits را فعال میکنیم. گزینه Volt. Ref ولتاژ مرجع ADC را مشخص میکند. حالت AREF برای استفاده از ولتاژ مرجع دلخواه روی پایه AREF میکرو است. حالت AVCC برای استفاده از ولتاژ تغذیه واحد ADC به عنوان ولتاژ مرجع است. حالت آخر که ما از آن استفاده میکنیم استفاده از ولتاژ مرجع داخلی واحد ADC است (2.56V). در این حالت چون ولتاژ روی AREF نیز ظاهر می شود باید از یک خازن 100nF روی پایه AREF استفاده کنیم. مقدار کلاک واحد را روی 500kHz می گذاریم. برای استفاده از LCD وارد تب Alphanumeric LCD می شویم و آن را Enable میکنیم. چون از LCD 16*2 استفاده میکنیم گزینه Characters/Line را برابر 16 میکنیم. ویزارد به صورت خودکار پورت A را به LCD اختصاص می دهد. چون پورت A به واحد ADC اختصاص دارد ما از پورت D استفاده می کنیم. بقیه تنظیمات را تغییر نمی دهیم. مطمئن میشویم تیک گزینه Program > Generate Code For Disabled Peripherals برداشته شده است. سپس گزینه Program > Generate, Save and Exit را میزنیم. در سه پنجره باز شده برای ذخیره کردن فایل های پروژه همه ی فایل ها را در یک فولدر و در کنار هم ذخیره میکنیم.

II. مشاهده می کنیم برخی خطوط به صورت خودکار به برنامه اضافه شده است. این خطوط وظیفه کنترل رجیسترهای مربوط به واحد ADC را بر عهده دارند. به کمک تابع read_adc() میتوان مقدار خوانده شده از واحد ADC را دریافت کرد. برای اندازه گیری دمای محیط از سنسور LM35 استفاده میکنیم. این سنسور دما را به شکل ولتاژ نشان می دهد. به ازای هر 1 درجه سانتیگراد خروجی سنسور 10mV تغییر میکند. در بالای تابع main متغیرهای Dvoltage, Avoltage و temp را از نوع float و آرایه char str[10] را تعریف می کنیم. بعد از تابع lcd_init(16)، مکان نمای LCD را به نقطه (0,0) می بریم و عبارت "Temp is:" را چاپ می کنیم. وارد حلقه ی بی نهایت می شویم. read_adc(0) ولتاژ را از کانال 0 واحد ADC یعنی PORTA0 دریافت می کند. این مقدار را در Dvoltageذخیره میکنیم. ابتدا باید ولتاژ دیجیتال را به ولتاژ آنالوگ تبدیل کنیم. از رابطه $V_{analog} = V_{digital} * \frac{V_{ref}}{2^n}$ برای تبدیل ولتاژ دیجیتال به آنالوگ استفاده می کنیم. n بیانگر وضوح بیتی واحد ADC (در اینجا ۸) و V_{ref} بیانگر ولتاژ مرجع واحد ADC (در اینجا 2.56V) است. بنابراین مقدار Dvoltage*2.56/256 را در Avoltage میریزیم. از آنجایی که خروجی سنسور برحسب میلی ولت است باید در 1000 ضرب شود و چون هر 10mV برابر ۱ درجه است باید تقسیم بر 10 شود. از این رو دما برابر با Avoltage*1000/10 است. این مقدار را در temp میریزیم. برای نمایش temp روی lcd باید آن را به شکل کاراکتری در بیاوریم. برای اینکار از تابع ftoa استفاده می کنیم. در بالای برنامه باید کتابخانه مربوطه یعنی stdlib.h را اضافه کنیم. با نوشتن دستور ftoa(temp,0,str) مقدار temp به صورت کاراکتری و بدون رقم اعشاری در str ذخیره می شود. حالا مکان نمای LCD را به خط دوم و ستون اول LCD هدایت میکنیم و با lcd_puts(str) و lcd_puts("^C") دما را چاپ می کنیم. از منوی Project > Build All برنامه را کامپایل میکنیم.

نحوه کار برنامه به این صورت است که ولتاژ خروجی سنسور LM35 را از کانال 0 واحد ADC میخواند و با انجام تبدیلات لازم دما را در LCD نمایش می دهد.

III. در پروتئوس یک پروژه جدید با تنظیمات پیش فرض ویزارد ایجاد میکنیم. (LM016L) 2*16 LCD، LM35، خازن و میکروکنترلر ATMEGA16 را با جستجو در Device ها پیدا میکنیم و به پروژه اضافه میکنیم. VSS را به زمین و VDD را به 5v+ متصل میکنیم. پایه VEE برای کنترل کنتراست کاراکتر ها استفاده می شود که میتوان با وصل کردن آن به پتانسیومتر مقدار آن را کنترل کرد. طبق سیم کشی ویزارد باید پایه های RS، RW و E را به PORTD0…2 وصل کنیم تا خود کدویژن عملیات کنترلی LCD را مدیریت کند. پایه های دیتا D0…3 را استفاده نمیکنیم و فقط پایه های D4…7 را به PORTD4…7 وصل میکنیم تا انتقال اطلاعات با کمک توابع کتابخانه ای کدویژن صورت پذیرد. AVREF را با خازن 100nF به زمین متصل میکنیم (در شبیه سازی ضروری نیست). AVCC را به 5V+ وصل میکنیم. پین ۱ سنسور را به 5V+، پین ۳ را به زمین و پین ۲ را به PORTA0 متصل میکنیم. روی میکرو دابل کلیک میکنیم. در پنجره باز شده بخش Program File کد هگز برنامه تولید شده در کدویژن را روی میکرو پروگرم میکنیم. با اجرای شبیه ساز مشاهده میکنیم که دمای سنسور روی LCD نمایش داده می شود. همچنین با تغییر دما بلافاصله روی LCD دمای نهایی چاپ می شود.

پیوست ها:

G:\Personal\Uni\Micro Lab\Micro\Az5-1\Az5-1.c

Notes | Az5-1.c

```
1  /*****************************************************
2  This program was created by the
3  CodeWizardAVR V3.12 Advanced
4  Automatic Program Generator
5  © Copyright 1998-2014 Pavel Haiduc, HP InfoTech s.r.l.
6  http://www.hpinfotech.com
7
8  Project : Session5-Experiment1
9  Version :
10 Date    : 10/21/2020
11 Author  : Ali Sepehrnia (96121038)
12 Company :
13 Comments:
14
15
16 Chip type               : ATmega16
17 Program type            : Application
18 AVR Core Clock frequency: 1.000000 MHz
19 Memory model            : Small
20 External RAM size       : 0
21 Data Stack size         : 256
22 *****************************************************/
23
24 #include <mega16.h>
25 #include <stdlib.h>
26 #include <delay.h>
27
28 // Alphanumeric LCD functions
29 #include <alcd.h>
30
31 // Declare your global variables here
32
33 // Voltage Reference: Int., cap. on AREF
34 #define ADC_VREF_TYPE ((1<<REFS1) | (1<<REFS0) | (1<<ADLAR))
```

G:\Personal\Uni\Micro Lab\Micro\Az5-1\Az5-1.c

Notes | Az5-1.c

```
36 // Read the 8 most significant bits
37 // of the AD conversion result
38 unsigned char read_adc(unsigned char adc_input)
39 {
40 ADMUX=adc_input | ADC_VREF_TYPE;
41 // Delay needed for the stabilization of the ADC input voltage
42 delay_us(10);
43 // Start the AD conversion
44 ADCSRA|=(1<<ADSC);
45 // Wait for the AD conversion to complete
46 while ((ADCSRA & (1<<ADIF))==0);
47 ADCSRA|=(1<<ADIF);
48 return ADCH;
49 }
50
51 void main(void)
52 {
53 // Declare your local variables here
54 float Dvoltage,Avoltage,temp;
55 char str[10];
56
57 // Input/Output Ports initialization
58 // Port A initialization
59 // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
60 DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
61 // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
62 PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);
63
64 // Port B initialization
65 // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
66 DDRB=(0<<DDB7) | (0<<DDB6) | (0<<DDB5) | (0<<DDB4) | (0<<DDB3) | (0<<DDB2) | (0<<DDB1) | (0<<DDB0);
67 // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
68 PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);
69
70 // Port C initialization
71 // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
```

G:\Personal\Uni\Micro Lab\Micro\Az5-1\Az5-1.c

Notes Az5-1.c

```
// Read the 8 most significant bits
// of the AD conversion result
unsigned char read_adc(unsigned char adc_input)
{
ADMUX=adc_input | ADC_VREF_TYPE;
// Delay needed for the stabilization of the ADC input voltage
delay_us(10);
// Start the AD conversion
ADCSRA|=(1<<ADSC);
// Wait for the AD conversion to complete
while ((ADCSRA & (1<<ADIF))==0);
ADCSRA|=(1<<ADIF);
return ADCH;
}

void main(void)
{
// Declare your local variables here
float Dvoltage,Avoltage,temp;
char str[10];

// Input/Output Ports initialization
// Port A initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);

// Port B initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRB=(0<<DDB7) | (0<<DDB6) | (0<<DDB5) | (0<<DDB4) | (0<<DDB3) | (0<<DDB2) | (0<<DDB1) | (0<<DDB0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);

// Port C initialization
```

G:\Personal\Uni\Micro Lab\Micro\Az5-1\Az5-1.c

Notes Az5-1.c

```
// Port C initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);

// Port D initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (0<<PORTD2) | (0<<PORTD1) | (0<<PORTD0);

// ADC initialization
// ADC Clock frequency: 500.000 kHz
// ADC Voltage Reference: Int., cap. on AREF
// ADC Auto Trigger Source: ADC Stopped
// Only the 8 most significant bits of
// the AD conversion result are used
ADMUX=ADC_VREF_TYPE;
ADCSRA=(1<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) | (0<<ADPS2) | (0<<ADPS1) | (1<<ADPS0);
SFIOR=(0<<ADTS2) | (0<<ADTS1) | (0<<ADTS0);

// Alphanumeric LCD initialization
// Connections are specified in the
// Project|Configure|C Compiler|Libraries|Alphanumeric LCD menu:
// RS - PORTD Bit 0
// RD - PORTD Bit 1
// EN - PORTD Bit 2
// D4 - PORTD Bit 4
// D5 - PORTD Bit 5
// D6 - PORTD Bit 6
// D7 - PORTD Bit 7
// Characters/line: 16
lcd_init(16);
lcd_gotoxy(0,0);
```

G:\Personal\Uni\Micro Lab\Micro\Az5-1\Az5-1.c

Notes | Az5-1.c

```
lcd_init(16);
lcd_gotoxy(0,0);
lcd_puts("Temp is:");

while (1)
      {
      // Place your code here
      Dvoltage=read_adc(0);
      Avoltage=Dvoltage*2.56/256;
      temp=Avoltage*1000/10;
      ftoa(temp,0,str);
      lcd_gotoxy(0,1);
      lcd_puts(str);
      lcd_puts("^C");
      }
}
```

